Vulnerability Assessment: **Insecure Direct Object Reference (IDOR) & API Information Disclosure**
Project 4: Web Application Security Assessment
Week 3: Secure SDLC and Code Review

Vulnerability Information

- **Vulnerability Type:** Insecure Direct Object Reference (IDOR) / Information Disclosure
- **Risk Level:** Critical
- **OWASP Category:** A01:2021 – Broken Access Control
- **Location:** /vulnerabilities/api/v2/user/

Vulnerable Code Analysis

- Low

```
function get_users() {
    const url = '/vulnerabilities/api/v2/user/';

    fetch(url, {
            method: 'GET',
        })
        .then(response => {
            if (!response.ok) {
                throw new Error('Network response was not ok');
        }
        return response.json();
        })
        .then(data => {
            loadTableData(data);
        })
        .catch(error => {
            console.error('There was a problem with your fetch operation:', error);
    });
}
```

- Medium

```
function get_user() {
    const url = '/vulnerabilities/api/v2/user/2';

    fetch(url, {
            method: 'GET',
        })
        .then(response => {
            if (!response.ok) {
                throw new Error('Network response was not ok');
        }
        return response.json();
        })
        .then(data => {
            update_username (data);
        })
        .catch(error => {
            console.error('There was a problem with your fetch operation:', error);
    });
}
```

- High

The API endpoints /vulnerabilities/api/v2/user/ and /vulnerabilities/api/v2/user/2 expose sensitive user data without authentication or authorization checks. Users can access or modify other users' information by changing the user ID parameter.
Additionally, the OpenAPI/Swagger documentation is publicly exposed, revealing endpoint details and potential attack vectors.

Secure Code Screenshot:

```javascript
// Example of a secure API call with token-based authentication and ownership validation
function get_my_profile() {
    const url = '/api/v2/my-profile';
    const token = sessionStorage.getItem('auth_token');

    fetch(url, {
        method: 'GET',
        headers: {
            'Authorization': `Bearer ${token}`
        }
    })
    .then(response => {
        if (!response.ok) throw new Error('Unauthorized or invalid token');
        return response.json();
    })
    .then(data => {
        displayUserProfile(data);
    })
    .catch(error => {
        console.error('Error fetching profile:', error);
    });
}
```

Mitigations Applied:

- Implemented authentication and authorization on all API endpoints
- Restricted access to authenticated users with valid tokens
- Added ownership validation on the server side
- Removed exposure of user enumeration endpoints (/user/)
- Restricted API documentation access to administrators only

Recommendations for Development Team

Immediate Actions:

1. **Authorization & Access Control**
    - Enforce authentication for all endpoints
    - Validate user ownership before returning data
    - Apply least privilege principles
2. **API Design & Architecture**
    - Replace numeric IDs with UUIDs
    - Use dedicated endpoints like /api/my-profile
    - Avoid exposing internal object IDs in responses
3. **Server-Side Validation**
    - Never trust client-provided identifiers
    - Implement Role-Based Access Control (RBAC)
    - Validate permissions for every request
4. **API Security Best Practices**
    - Use OAuth2 or JWT for session management
    - Apply rate limiting to prevent enumeration attacks
    - Restrict access to OpenAPI/Swagger documentation
    - Conduct periodic penetration tests

References

- **OWASP Cheat Sheet:** API Security Cheat Sheet
- **CWE Reference:** CWE-639: Authorization Bypass Through User-Controlled Key
- **Best Practice:** Enforce authentication, authorization, and object-level access control for all API endpoints.